

Módulo 1: Fundamentos de Python y Lógica Algorítmica

Introducción

Programar para Ciencia de Datos comienza antes de usar una biblioteca: hay que convertir una pregunta en pasos que una computadora pueda ejecutar y verificar.

En este módulo aprenderás a:

- representar información con estructuras de datos;
- controlar el flujo de un programa con decisiones y repeticiones;
- descomponer un problema en funciones pequeñas;
- reconocer cuándo una solución recursiva es clara y cuándo conviene una solución iterativa;
- validar tu lógica con casos de prueba.

Reto del módulo

Pregunta guía: ¿Cómo podemos construir un algoritmo que reconozca patrones en textos sin depender de herramientas externas?

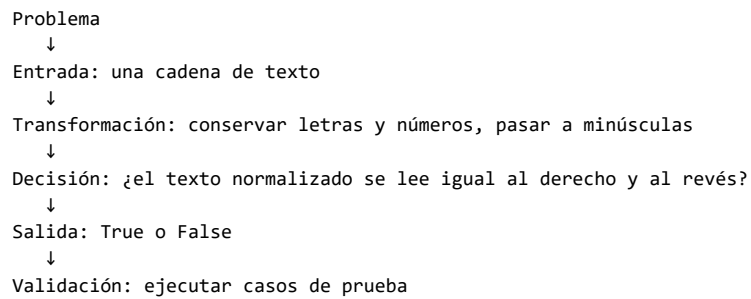
Producto final: una función que determine si una cadena es un palíndromo ignorando espacios, signos y diferencias entre mayúsculas y minúsculas.

Criterios de éxito:

- la función recibe una cadena y devuelve True o False;
- normaliza el texto antes de compararlo;
- funciona con frases, espacios, signos y mayúsculas;
- incluye pruebas con casos verdaderos y falsos;
- la solución puede explicarse paso a paso.

1. Del problema al algoritmo

Usa este mapa antes de escribir código:



Pseudocódigo

```
recibir texto
normalizar texto
comparar texto con su versión invertida
si son iguales:
    devolver verdadero
si no:
    devolver falso
```

Comprobación rápida

Antes de programar, prueba mentalmente:

Entrada	Texto normalizado	Resultado
"reconocer"	reconocer	True
"Anita lava la tina"	anitalavalatina	True
"Python"	python	False

2. Variables y tipos de datos

Una variable es un nombre asociado a un valor.

```
nombre = "Ada"  
edad = 36  
es_investigadora = True
```

Tipos básicos:

Tipo	Ejemplo	Uso
str	"datos"	texto
int	42	cantidades enteras
float	3.14	medidas decimales
bool	True	decisiones
None	None	ausencia de valor

Comprueba el tipo con `type(valor)`. Elige nombres que expliquen qué representa cada dato.

Conversión de tipos

```
cantidad = int("12")  
precio = float("4.50")  
mensaje = str(2026)
```

La conversión puede fallar si el contenido no tiene el formato esperado. Por eso conviene validar entradas externas.

3. Estructuras de datos

Una estructura de datos organiza valores para poder consultarlos y modificarlos con claridad.

Listas: colección ordenada y modificable

```
sensores = ["temperatura", "humedad", "luz"]
sensores.append("presion")
primer_sensor = sensores[0]
```

Características:

- conserva el orden;
- permite valores repetidos;
- se recorre con for;
- usa índices desde 0.

Operaciones frecuentes:

```
len(sensores)          # cantidad de elementos
"luz" in sensores       # comprobar pertenencia
sensores[1:3]          # obtener una sección
```

Tuplas: colección ordenada e inmutable

```
coordenada = (4.5, -74.1)
latitud, longitud = coordenada
```

Usa una tupla cuando los valores formen una unidad que no debería modificarse accidentalmente.

Conjuntos: valores únicos

```
categorias = {"agua", "aire", "agua"}
# categorias queda como {"agua", "aire"}
```

Los conjuntos son útiles para eliminar duplicados y comparar grupos.

```
nuevas = {"agua", "suelo"}
comunes = categorias & nuevas
```

Diccionarios: pares clave-valor

```
muestra = {
    "id": "M-001",
    "temperatura": 21.5,
    "valida": True,
}

muestra["temperatura"]
muestra["humedad"] = 64.2
```

Un diccionario funciona bien cuando cada dato tiene una etiqueta. Las claves deben ser únicas.

Para evitar errores al consultar una clave opcional:

```
humedad = muestra.get("humedad", 0.0)
```

¿Qué estructura elegir?

Necesidad	Estructura
Secuencia ordenada que cambia	lista
Registro pequeño que no cambia	tupla
Valores sin duplicados	conjunto
Datos identificados por etiquetas	diccionario

4. Flujo de control

Condicionales

```
if temperatura > 30:
    estado = "alta"
elif temperatura < 10:
    estado = "baja"
else:
    estado = "normal"
```

Una condición debe representar una pregunta que pueda responderse como True o False.

Repetición con for

```
mediciones = [18.2, 20.1, 19.8]

for medicion in mediciones:
    print(medicion)
```

Usa for cuando conoces la colección o el rango que quieres recorrer.

Repetición con while

```
intentos = 0

while intentos < 3:
    print("Intento", intentos + 1)
    intentos += 1
```

Usa while cuando la repetición depende de una condición. Actualiza la variable de control para evitar un ciclo infinito.

5. Funciones y descomposición

Una función encapsula una tarea con entradas y una salida.

```
def convertir_celsius_a_fahrenheit(celsius: float) -> float:
    """Convierte una temperatura de Celsius a Fahrenheit."""
    return celsius * 9 / 5 + 32
```

Buenas prácticas:

1. una función debe tener una responsabilidad clara;
2. los nombres deben describir la acción o el resultado;
3. documenta entradas y salida cuando la función no sea obvia;
4. devuelve valores en lugar de imprimirlos si otra parte del programa los necesita;
5. prueba casos normales y casos límite.

Caso límite

Para el reto del módulo, considera también:

- cadena vacía: "";
- cadena de un solo carácter;
- texto formado solo por signos o espacios;
- caracteres con mayúsculas;
- frases con signos de puntuación.

Define qué comportamiento esperas antes de implementarlo.

6. Recursividad

Una función recursiva se llama a sí misma para resolver una versión más pequeña del mismo problema.

Toda recursión necesita dos piezas:

1. caso base: detiene las llamadas;
2. caso recursivo: reduce el problema y vuelve a llamar a la función.

Ejemplo: cuenta regresiva

```
def cuenta_regresiva(numero: int) -> None:
    """Muestra los números desde numero hasta cero."""
    if numero < 0: # caso base
        return

    print(numero)
    cuenta_regresiva(numero - 1) # caso recursivo
```

Flujo para `cuenta_regresiva(2)`:

```
cuenta_regresiva(2)
-> cuenta_regresiva(1)
  -> cuenta_regresiva(0)
    -> cuenta_regresiva(-1) -> termina
```

Si no reduces el problema o no alcanzas el caso base, Python puede producir un error por exceso de profundidad de recursión.

Ejemplo: suma de una lista

```
def sumar_recursivo(valores: list[int]) -> int:
    """Devuelve la suma de una lista usando recursividad."""
    if not valores:
        return 0
    return valores[0] + sumar_recursivo(valores[1:])
```

La recursividad puede ser expresiva para árboles, carpetas o problemas que se dividen en subproblemas. Para recorridos lineales simples, un ciclo suele consumir menos memoria y ser más fácil de depurar.

Recursividad e iteración

Pregunta	Recursividad	Iteración
¿Cómo repite?	llamadas a sí misma	for o while
¿Qué necesita?	caso base	condición de salida
Riesgo típico	demasiadas llamadas	ciclo infinito
Uso recomendable	estructuras anidadas	recorridos lineales

El reto del archivo `02_reto_algoritmico.py` se puede resolver de forma iterativa o usando dos índices. Primero implementa la versión más clara; después compara alternativas.

7. Validación y depuración

Un `assert` comprueba que una expresión sea verdadera.

```
assert 2 + 2 == 4
assert convertir_celsius_a_fahrenheit(0) == 32
```

Si la expresión es falsa, Python muestra un `AssertionError`. Lee el caso que falló y revisa la transformación de los datos.

Proceso recomendado:

1. Ejecutar una prueba pequeña
2. Leer el valor obtenido
3. Compararlo con el esperado
4. Revisar una sola transformación
5. Repetir con un caso límite

No elimines los `asserts` cuando termines: son una evidencia rápida de que el algoritmo conserva su comportamiento.

8. Actividad del estudiante

Abre `02_reto_algoritmico.py` y completa la función `es_palindromo`.

Reto 1: reproducción

Haz que pasen los casos incluidos en el archivo.

Reto 2: adaptación

Añade una prueba para una frase relacionada con Ciencia de Datos, por ejemplo:

```
assert es_palindromo("A mamá Roma le aviva el amor a mamá") is True
```

Reto 3: extensión

Crea una segunda función que explique por qué una cadena no es palíndroma mostrando el primer par de caracteres diferentes.

No cambies la función original hasta que todos sus asserts pasen.

Evidencia y evaluación

Criterio	Logrado cuando...
Descomposición	explicas entrada, transformación, decisión y salida
Estructuras	eliges una estructura adecuada y justificas la elección
Función	recibe datos, devuelve un resultado y tiene una responsabilidad
Lógica	normaliza y compara sin depender de casos concretos
Validación	los asserts cubren casos verdaderos, falsos y límite
Comunicación	puedes explicar el algoritmo con pseudocódigo

Reflexión final

Responde en tres líneas:

1. ¿Qué estructura de datos usarías para guardar muchas mediciones de un sensor? ¿Por qué?
2. ¿Cuál es el caso base de una solución recursiva?
3. ¿Qué caso de prueba descubrió un error o confirmó tu lógica?